

PLSQL- Suite-

Sanae MAZOUZ

LES EXCEPTIONS

Les Exceptions

Afin d'éviter qu'un programme s'arrête à la première erreur :

- **Requête ne retournant aucune ligne,**
- **Valeur incorrecte à écrire dans la base,**
- **Conflit de clés primaires,**
- **Division par zéro, etc...**

Il est indispensable de prévoir tous les cas potentiels d'erreurs et d'associer à chacun de ces cas la programmation d'une exception PL/SQL.

Les Exceptions

Les exceptions peuvent se programmer dans :

- **Un bloc PL/SQL**
- **Un sous-programme (fonction ou procédure cataloguée)**
- **Un paquetage**
- **Un déclencheur**

Les Exceptions

Une exception PL/SQL correspond à une condition d'erreur et est associée à un identificateur.

Une exception est détectée (aussi dite « levée ») au cours de l'exécution d'une partie de programme (entre un BEGIN et un END).

Une fois levée :

- **l'exception termine le corps principal des instructions**
- **et renvoie au bloc EXCEPTION du programme en question**

Les Exceptions

La syntaxe générale d'un bloc d'exceptions est la suivante:

```
EXCEPTION
```

```
    WHEN exception1 [OR exception2 ...] THEN  
        instructions;
```

```
    [WHEN exception3 [OR exception4 ...] THEN  
        instructions; ]
```

```
    [WHEN OTHERS THEN  
        instructions; ]
```

Les Exceptions

PL/SQL offre au développeur un mécanisme de gestion des exceptions. Il permet de préciser la logique du traitement des erreurs survenues dans l'un de ses blocs.

Il existe deux types d'exceptions :

- **Erreur ORACLE**
- **Erreur utilisateur**

Les Exceptions

Exceptions définies par ORACLE

- Nommées par Oracle,
- Se déclenchent automatiquement,
- Nécessitent de prévoir la prise en compte de l'erreur dans la section EXCEPTION.

Exception prédéfinie	Erreur Oracle	Valeur de SQLCODE
ACCESS_INTO_NULL	ORA-06530	-6530
CASE_NOT_FOUND	ORA-06592	-6592
COLLECTION_IS_NULL	ORA-06531	-6531
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
NOT_LOGGED_ON	ORA-01012	-1012
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
SELF_IS_NULL	ORA-30625	-30625
STORAGE_ERROR	ORA-06500	-6500
SUBSCRIPT_BEYOND_COUNT	ORA-06533	-6533
SUBSCRIPT_OUTSIDE_LIMIT	ORA-06532	-6532
SYS_INVALID_ROWID	ORA-01410	-1410
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
VALUE_ERROR	ORA-06502	-6502
ZERO_DIVIDE	ORA-01476	-1476

Les Exceptions

Exceptions prédéfinies

- **ZERO_DIVIDE** : tentative de division par zéro.
- **TOO_MANY_ROWS** : la commande ***SELECT INTO*** retourne plus d'une ligne.
- **NO_DATA_FOUND** : déclenchée si la commande ***SELECT INTO*** ne retourne aucune ligne ou si l'on fait référence à un enregistrement ,non initialisé, d'un tableau PL/SQL.
- **LOGIN_DENIED** : connexion à la base est échouée, car le nom utilisateur ou le mot de passe est invalide.
- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert.
-

Les Exceptions

Exceptions définies par l'utilisateur

- Nommés par le programmeur
- Arrêt de l'exécution du bloc
- Sont déclenchées par une instruction du programme soit automatiquement ou manuellement
- Nécessitent de prévoir la prise en compte de l'erreur dans la section EXCEPTION

Oracle fournit les fonctions **SQLCODE** et **SQLERRM**, qui renvoient respectivement le code et le message d'erreur correspondant à l'exception.

Les Exceptions

- Si une anomalie se produit, le bloc **EXCEPTION** s'exécute.
- Si le programme prend en compte l'erreur dans une entrée **WHEN...**, les instructions de cette entrée sont exécutées et le programme se termine.
- Si l'exception n'est pas prise en compte dans le bloc **EXCEPTION** :
 - il existe une section **OTHERS** où des instructions s'exécutent ;
 - S'il n'existe pas une section **OTHERS**, l'exception sera propagée au programme appelant

Les Exceptions

Syntaxe erreur ORACLE prédéfinie :

Certaines erreurs ORACLE ont déjà un nom. Il est donc inutile de les déclarer comme précédemment. On utilise leur nom dans la section EXCEPTION.

Exemple :

```
DECLARE
    ...
BEGIN
    ...
EXCEPTION
    WHEN      nom_erreur      THEN...traitement de l'erreur
END;
```

Les Exceptions

Exemple :

```
DECLARE
    salaire emp.sal%TYPE ;
BEGIN
    SELECT sal into salaire FROM emp WHERE empno =10;
    .....
EXCEPTION
    WHEN TOO_MANY_ROWS THEN traitements ;
    WHEN NO_DATA_FOUND THEN traitements ;
END ;
```

Les Exceptions

Fonctionnement :

Lorsqu'une exception est détectée, il y a :

- Arrêt de l'exécution du bloc
- Branchement sur la section exception
- Parcours des clauses WHEN jusqu'au bon choix
- Exécution des instructions associées
- Une fois le traitement de l'erreur est terminé, c'est le bloc suivant qui est effectué

Les Exceptions utilisateur

- Il est possible de définir ses propres exceptions
- *Deux étapes essentielles :*

Déclaration:

- La déclaration du nom de l'exception doit se trouver dans la section déclarative du sous-programme.
- *Nom_Exception* **EXCEPTION;**

Déclenchement

- Dans ce cas, le programme doit explicitement rediriger le traitement vers le bloc des exceptions par la directive **RAISE.**

Les Exceptions utilisateur

Syntaxe :

```
DECLARE  
  
    .....  
  
    Nom_exception EXCEPTION  
BEGIN  
  
    .....--Instructions  
If (condition_erreur) THEN RAISE Nom_exception;  
    .....  
EXCEPTION  
    WHEN Nom_exception THEN traitements;  
END;
```

*On nomme
l'erreur*

*On déclenche
l'erreur*

*On traite
l'erreur*

Remarque :

On sort du bloc après l'exécution du traitement d'erreur.

DECLARE

CURSOR emp_cur IS SELECT ename, sal FROM emp WHERE deptno = 10;

v_ename emp.ename%TYPE;

v_sal emp.sal%TYPE;

ERR_salaire EXCEPTION;

BEGIN

OPEN emp_cur;

FETCH emp_cur INTO v_ename, v_sal; --premier enregistrement

WHILE emp_cur%FOUND LOOP -- S'il y a un enregistrement récupéré

IF v_sal IS NULL THEN

RAISE ERR_salaire;

..... --Traitement de l'enregistrement récupéré

FETCH emp_cur INTO v_ename, v_sal; --enregistrement suivant

END LOOP;

CLOSE emp_cur;

EXCEPTION

WHEN ERR_salaire THEN INSERT INTO temp VALUES (v_ename, 'Salaire non défini');

WHEN NO_DATA_FOUND THEN INSERT INTO temp VALUES (v_ename, 'Employé non trouvé');

END;

Les Exceptions utilisateur

Exception 'OTHERS'

Le module **EXCEPTION** permet également de traiter un code erreur qui n'est traité par aucune des exceptions. Le nom générique de cette exception (prédéfinie) est **OTHERS**.

Syntaxe :

```
EXCEPTION
    WHEN exception1 THEN
        traitement1;
    WHEN exception2 THEN
        traitement2;
    WHEN OTHERS THEN
        traitement3;
```

Deux fonctions permettent de récupérer des informations sur l'erreur Oracle :

- **Sqlcode** : retourne une valeur numérique: numéro de l'erreur.
- **Sqlerrm** : renvoie le libellé de l'erreur.

Les Exceptions utilisateur

Remarque :

Sqlcode : varie de **-20999** à **-20000**

c'est la plage réservée au développeur pour définir ses propres erreurs avec ***RAISE_APPLICATION_ERROR***

Plage	Signification
0	Pas d'erreur
+100	NO_DATA_FOUND
-1 à -19999	Erreurs Oracle prédéfinies
-20000 à -20999	Erreurs personnalisées développeur
-21000 et au-delà	Erreurs Oracle internes

```

SET SERVEROUTPUT ON;

DECLARE
    v_nom emp.ename%TYPE;
    v_sal emp.sal%TYPE;
BEGIN
    SELECT ename, sal INTO v_nom, v_sal      FROM emp      WHERE empno
= 7839;
    DBMS_OUTPUT.PUT_LINE('Employé : ' || v_nom);
    DBMS_OUTPUT.PUT_LINE('Salaire : ' || v_sal);

    IF v_sal > 4000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salaire supérieur à la
limite autorisée');
    END IF;

    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur détectée !');
            DBMS_OUTPUT.PUT_LINE('SQLCODE : ' || SQLCODE);
            DBMS_OUTPUT.PUT_LINE('SQLERRM : ' || SQLERRM);

END;
/

```

Les Exceptions utilisateur

Résultat :

Employé : KING

Salaire : 5000

Erreur détectée !

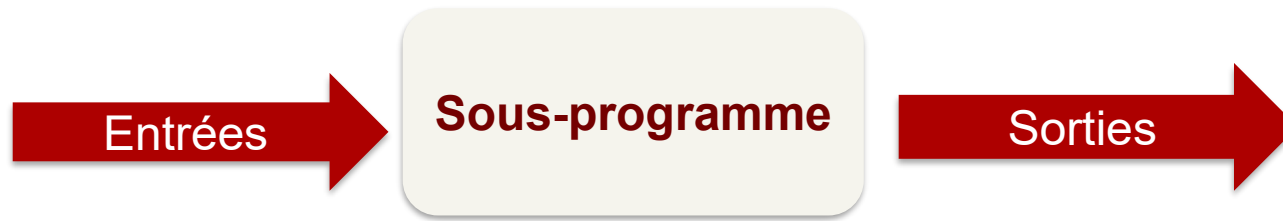
SQLCODE : -20001

SQLERRM : ORA-20001: Salaire supérieur à la
limite autorisée

LES SOUS-PROGRAMMES

Les Sous-programmes

- Les sous-programmes sont des blocs PL/SQL nommés et capables d'inclure des paramètres en entrée et en sortie.



- Dans PL/SQL:

Il existe deux types de sous-programmes :

- les procédures
 - et les fonctions
- les procédures réalisent des actions
 - les fonctions retournent un unique résultat

Les Sous-programmes

Les avantages :

- **Sécurité** : les droits d'accès ne portent plus sur des objets (table, vue, variable...) mais sur des programmes stockés.
- **Intégrité** : les traitements dépendants sont exécutés dans le même bloc (transactions).
- **Performance** : réduction du nombre d'appels à la base (utilisation d'un programme partagé).
- **Productivité** : simplicité de la maintenance des programmes (modularité, extensibilité, réutilisabilité) notamment par l'utilisation de paquetages.

Les sous-programmes

- ✓ Une procédure ou une fonction, peut être appelée à l'aide de :
 - L'interface de commande SQL*Plus (commande EXECUTE).
 - Dans un programme externe (Java, C...).
 - Par d'autres procédures ou fonctions.
 - Ou dans le corps d'un déclencheur.
- ✓ Les fonctions peuvent être appelées dans une instruction SQL (SELECT, INSERT, et UPDATE).
- ✓ Le cycle de vie d'un sous-programme est le suivant :
 - Création de la procédure ou fonction (compilation et stockage)
 - Appels
 - Et éventuellement suppression du sous-programme de la base

Les Procédures

- Pour créer une procédure dans son propre schéma, le privilège ***CREATE PROCEDURE*** est requis.
- Pour créer une procédure dans un autre schéma, il faut posséder le privilège ***CREATE ANY PROCEDURE***.

Les sous-programmes ont une partie :

- de déclaration des variables,
- des instructions
- et une dernière pour gérer les exceptions (erreurs produites durant l'exécution).

Les Procédures

- Une procédure est paramétrable afin d'en faciliter la réutilisation.
- Une procédure est appelée :
 - Dans un bloc PL/SQL :
`nom_Procédure[(liste paramètres)]`
 - Dans SQL*Plus :
`SQL> execute nom_Procédure[(liste paramètres)];`

Les Procédures

Syntaxe

```
CREATE [OR REPLACE] PROCEDURE
nom_proc
(param1 [IN |OUT|INOUT],
param2 [IN |OUT|INOUT] ...)
IS|AS
-- Déclarations locales
BEGIN
-- Instructions
[EXCEPTION
-- Traitement des exceptions]
END;
```

Les Procédures

Au niveau de la liste d'arguments :

Les arguments sont précédés des mots réservés **IN**, **OUT**, ou **IN OUT** et ils sont séparés par des virgules,

- **IN** : paramètre "entrant". il s'agit d'un paramètre dont la valeur est fournie à la procédure stockée. Cette valeur sera utilisée pendant la procédure.
- **OUT** : paramètre "sortant", dont la valeur va être établie au cours de la procédure et qui pourra ensuite être utilisé en dehors de cette procédure.
- **IN OUT** : un tel paramètre sera utilisé pendant la procédure, verra éventuellement sa valeur modifiée par celle-ci, et sera ensuite utilisable en dehors.

Les Procédures

Remarques :

- Contrairement au PL/SQL, la zone de déclaration n'a pas besoin d'être précédé du mot réservé **DECLARE**. Elle se trouve entre le **IS** et le **BEGIN**.
- La dernière ligne de chaque procédure doit être composée du seul caractère **/** pour spécifier au moteur le déclenchement de son exécution.

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE modifie_salaire (id IN NUMBER,
taux IN NUMBER)
IS
BEGIN
    UPDATE emp SET sal = sal * (1 + taux) WHERE empno = id;
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Employé inexistant');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Salaire modifie avec succès');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;

/
```


Les Procédures

Compilation de la procédure « *modifie_salaire* »

Il faut compiler le fichier SQL qui s'appelle ici *modifie_salaire.sql* (le nom du script, dans cet exemple, correspond à celui de la procédure; il faut donc passer, en argument, le nom du script sql à la commande *start* ou bien la commande *@*).

Exemple :

```
SQL> start modifie_salaire  
Procedure created.
```

Les Procédures

Exécution de la procédure

```
SQL> BEGIN
      modifie_salaire (15,1);
      END;
      /
      PL/SQL procedure successfully completed.
```

Les Procédures

- ***Modification d'une procédure***

Si la base de données évolue, il faut recompiler les procédures existantes pour qu'elles tiennent compte de ces modifications.

```
ALTER PROCEDURE nom_procedure COMPILE;
```

- ***Exemple :***

```
ALTER PROCEDURE modifie_salaire COMPILE;
```

Les Procédures

- ***Suppression d'une procédure***

```
DROP PROCEDURE nom_procedure;
```

- ***Exemple :***

```
DROP PROCEDURE modifie_salaire;
```

Les Fonctions

- Une fonction est une procédure qui retourne une valeur. La seule différence syntaxique par rapport à une procédure se traduit par la présence du mot clé ***RETURN.***
- Une fonction précise le type de donnée qu'elle retourne dans son prototype (signature de la fonction).

Les Fonctions

Syntaxe

```
CREATE [OR REPLACE] FUNCTION nom_func
    (param1 [IN |OUT|INOUT],
     param2 [IN |OUT|INOUT] ...)
RETURN type_valeur_de_retour
IS|AS
-- Déclarations locales
BEGIN
-- Instructions
RETURN valeur_de_retour;
[EXCEPTION
-- Traitement des exceptions]
END;
```

Les Fonctions

Exemple :

```
CREATE OR REPLACE FUNCTION get_salaire (p_empno IN NUMBER)
RETURN NUMBER
IS
    v_sal emp.sal%TYPE;
BEGIN
    SELECT sal INTO v_sal
    FROM emp
    WHERE empno = p_empno;

    RETURN v_sal;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20010, 'Employé inexistant');
END;
/
```

Compilation

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      DBMS_OUTPUT.PUT_LINE('Salaire = ' || get_salaire(7369));
  3  END;
  4  /
Salaire = 2400

Procédure PL/SQL terminée avec succès.

SQL> SELECT get_salaire(7369) FROM dual;

GET_SALAIRE(7369)
-----
                2400

SQL> |
```


Compilation

- Si le message suivant apparaît :

Avertissement : « *Fonction/Procédure créée avec erreurs de compilation* ».

Deux techniques peuvent être utilisées pour visualiser les erreurs de compilation :

- Faire SHOW ERRORS sous SQL*Plus
- Interroger la vue USER_ERRORS :

```
SELECT LINE,POSITION,TEXT FROM USER_ERRORS WHERE  
NAME='nomFonction/nomProcédure';
```

*Sinon, si le message « *Fonction/Procédure créée.* » apparaît,*

→ le sous-programme est correctement compilé et stocké en base.

Compilation

```
SQL> @F:\moyenne_salaire.sql
```

```
Avertissement : Fonction créée avec erreurs de compilation.
```

```
SQL> show errors
```

```
Erreurs pour FUNCTION MOYENNE_SALAIRE :
```

```
LINE/COL ERROR
```

```
-----  
6/4      PL/SQL: SQL Statement ignored
```

```
6/54     PL/SQL: ORA-00904: "V_ID_EMP" : identificateur non valide
```

Récurtivité

- La récursivité est permise dans PL/SQL.
- Comme dans tout programme récursif, il ne faut pas oublier la condition de terminaison !

Récurtivité

Code PL/SQL	Commentaires
<pre>CREATE FUNCTION factorielle(n POSITIVE) RETURN INTEGER IS BEGIN IF n = 1 THEN RETURN 1; ELSE RETURN n * factorielle(n - 1); END IF; END factorielle; /</pre>	Condition de terminaison. Appel récursif.
<pre>SQL> SELECT factorielle(30) "Factorielle 30" FROM DUAL; Factorielle 30 ----- 2,6525E+32</pre>	Appel de la fonction.

Recompilation d'une fonction

- Oracle recompile automatiquement un sous-programme quand un objet qui en dépend directement (table, vue, synonyme, séquence, etc.) a été modifié dans sa structure.
- La syntaxe suivante permet de recompiler manuellement une fonction :

ALTER FUNCTION nomFonction COMPILE;

Destruction d'un sous-programme

- Pour supprimer une fonction dans un autre schéma, le privilège **DROP ANY PROCEDURE** est requis.
- **DROP FUNCTION** [*schéma.*]*nomFonction*;

Il ne faut pas utiliser cette commande pour enlever une procédure ou une fonction d'un paquetage. Pour cela, nous verrons qu'il faudra redéfinir la spécification et le corps du nouveau paquetage en utilisant la directive : **OR REPLACE**

LES PAQUETAGES

Les Paquetages (packages)

- Un paquetage (*package*) est un composant qui regroupe *plusieurs objets (variables, exceptions, curseurs, fonctions, procédures, etc.)* formant un ensemble de services homogènes.
- Un paquetage est un ensemble de procédures et fonctions regroupées dans un objet nommé.
- Par exemple le paquetage Oracle DBMS_LOB regroupe toutes les fonctions et procédures manipulant les grands objets (LOBs).
- Le paquetage UTL_FILE regroupe les procédures et fonctions permettant de lire et écrire des fichiers du système d'exploitation

Le Package

Un paquetage est organisé en deux parties distinctes :

- **Une partie spécification** : Elle permet de spécifier à la fois les fonctions et procédures publiques ainsi que les déclarations des types, variables, constantes, exceptions et curseurs utilisés dans le paquetage et visibles par le programme appelant.
- **Une partie corps** : Elle contient les blocs et les spécifications de tous les objets publics listés dans la partie spécification. Cette partie peut inclure des objets qui ne sont pas listés dans la partie spécification, et sont donc privés. Cette partie peut également contenir du code qui sera exécuté à chaque invocation du paquetage par l'utilisateur.

Spécification

- La syntaxe simplifiée de la déclaration de la spécification d'un paquetage (**CREATE PACKAGE**) est la suivante :

```
CREATE [OR REPLACE] PACKAGE nom_package
AS
    [-- Définition de types]
    [-- Spécification de curseurs]
    [-- Déclaration de variables]
    [-- Déclaration de sous-programmes]
END;
```

Implémentation

- La syntaxe simplifiée de l'implémentation d'un paquetage (**CREATE PACKAGE BODY**) est la suivante :

```
CREATE [OR REPLACE] PACKAGE BODY nom_package
AS
    [-- Définition de types]
    [-- Spécification de curseurs]
    [-- Déclaration de variables]
    [-- Définition de sous-programmes]
END;
```

Exemple

Spécification

```
CREATE PACKAGE GestionEmployes AS

    FUNCTION NbEmployes(pdeptno IN NUMBER)
    RETURN NUMBER;

    PROCEDURE AugmenterSalaire(pempno IN NUMBER,
                                ppct    IN NUMBER);

END GestionEmployes;
/
```

```

CREATE PACKAGE BODY GestionEmployes AS

    FUNCTION NbEmployes(pdeptno IN NUMBER)
    RETURN NUMBER IS
        resultat NUMBER;
    BEGIN
        SELECT COUNT(*) INTO resultat
        FROM EMP
        WHERE DEPTNO = pdeptno;
        RETURN resultat;
    END NbEmployes;

    PROCEDURE AugmenterSalaire(pempno IN NUMBER,
                                ppct    IN NUMBER) IS
    BEGIN
        UPDATE EMP
        SET SAL = SAL + (SAL * ppct / 100)
        WHERE EMPNO = pempno;
    END AugmenterSalaire;

END GestionEmployes;
/

```

Appel

- L'accès à un sous-programme « *sp* » d'un paquetage « *paq* » s'écrit *paq.sp*.
- L'appel de ce sous-programme suit les mêmes règles que celles étudiées dans les sections précédentes.

LES TRIGGERS

Un déclencheur

- Un trigger permet de spécifier les réactions du système d'information lorsque l'on « touche » à ses données.
- Concrètement il s'agit de définir un traitement (un bloc PL/SQL) à réaliser lorsqu'un événement survient
- Déclencheur :
 - **Action** ou ensemble d'actions déclenchée(s) **automatiquement** lorsqu'une **condition** se trouve satisfaite après l'apparition d'un **événement**.

Un déclencheur

Un déclencheur est une règle **ECA** :

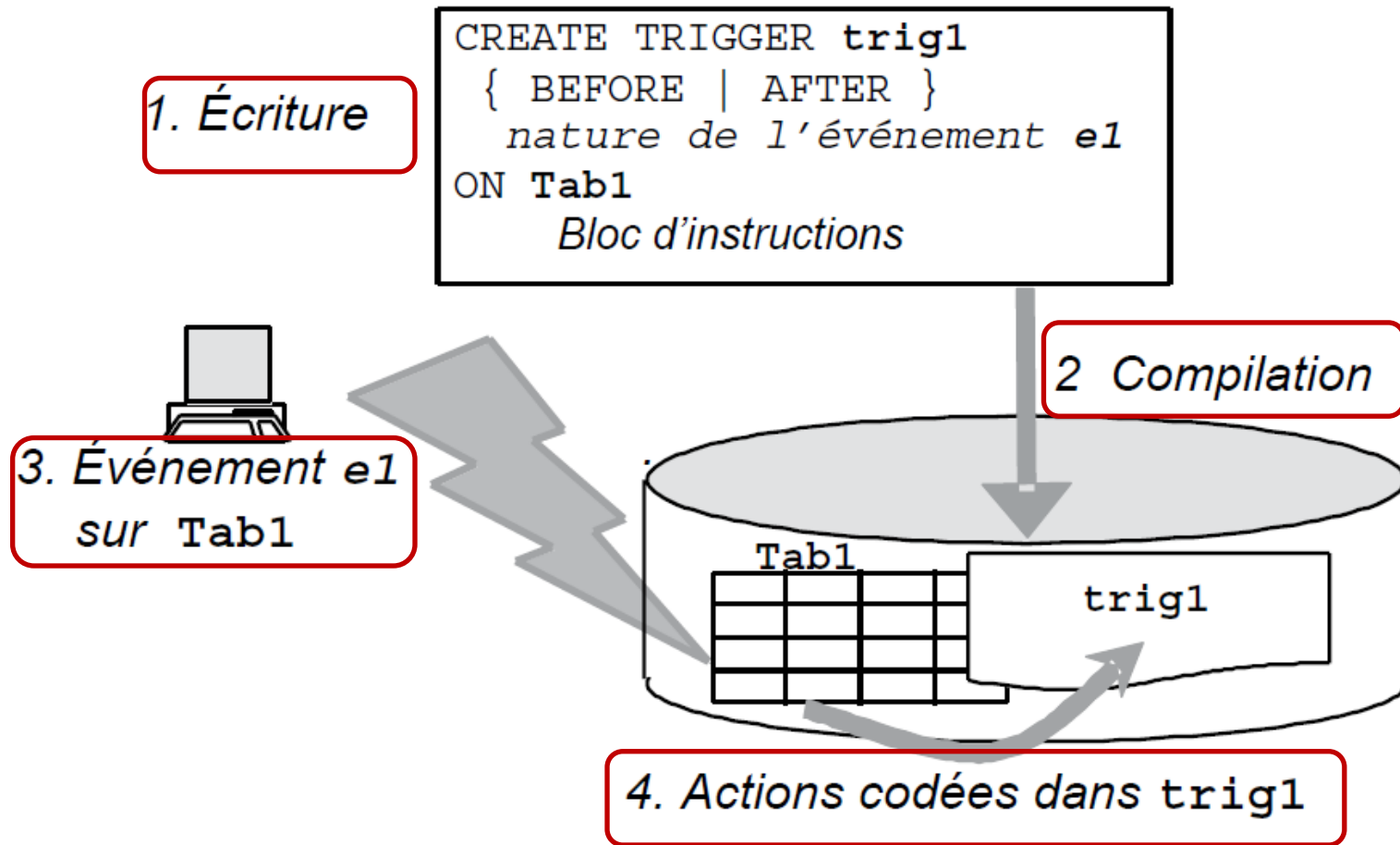
- **Événement** = mise à jour, suppression, insertion
- **Condition** = expression logique vraie ou fausse, optionnelle
- **Action** = procédure exécutée lorsque la condition est vérifiée.

À quoi sert un déclencheur ?

Un déclencheur permet de :

- **Programmer toutes les règles de gestion qui n'ont pas pu être mises en place par des contraintes au niveau des tables.**
- **Déporter des contraintes au niveau du serveur et alléger ainsi la programmation client.**

Mécanisme général



Triggers

Utilité des triggers

- Renforcer la cohérence des données d'une façon transparente pour le développeur,
- Propagation des MAJ dans une base de données distribuée.
- Sécurité...

Privilège

- Pour pouvoir créer un déclencheur, il faut disposer du privilège CREATE TRIGGER.
- Pour créer un déclencheur dans un autre schéma, le privilège CREATE ANY TRIGGER est requis.

Triggers

Remarques :

- Avec ORACLE, les événements sont prédéfinis et les conditions et les actions sont spécifiées par le langage procédural PL/SQL.
- Les événements sont de six types et ils peuvent porter sur des tables ou des colonnes :
 - BEFORE INSERT
 - AFTER INSERT
 - BEFORE UPDATE
 - AFTER UPDATE
 - BEFORE DELETE
 - AFTER DELETE

Triggers

Syntaxe

```
CREATE [ OR REPLACE ] TRIGGER  nom_trigger
  { BEFORE | AFTER }           // événement
  { INSERT | DELETE | UPDATE [ OF liste de
colonnes ] }
  ON table
  [ FOR EACH ROW ]
  [ WHEN ( condition de déclenchement ) ] //
condition
  corps (bloc PL/SQL)
```

Triggers

Entête du trigger

On définit :

- la table à laquelle le trigger est lié.
- les instructions du LMD qui déclenchent le trigger.
- le moment où le trigger va se déclencher par rapport à l'instruction LMD (avant ou après).
- si le trigger se déclenche une seule fois pour toute l'instruction (trigger d'état), ou une fois pour chaque ligne modifiée/insérée/supprimée (trigger ligne, avec l'option FOR EACH ROW).
- et éventuellement une condition supplémentaire de déclenchement (clause WHEN).

Triggers

Type de trigger

Nature de l'événement	État (<i>statement trigger</i>) sans FOR EACH ROW	Ligne (<i>row trigger</i>) avec FOR EACH ROW
BEFORE	Exécution une fois avant la mise à jour.	Exécution avant chaque ligne mise à jour.
AFTER	Exécution une fois après la mise à jour.	Exécution après chaque ligne mise à jour.

Statement trigger

- Exemple : si un UPDATE touche 1000 lignes → le trigger tourne 1 fois.

Row trigger

- Exemple : si un UPDATE touche 1000 lignes → le trigger tourne 1000 fois.

Les Déclencheurs de Lignes

- Un déclencheur de lignes est déclaré avec la directive **FOR EACH ROW**.
- Ce n'est que dans ce type de déclencheur qu'on a accès aux *anciennes valeurs (OLD)* et aux *nouvelles valeurs (NEW)* des colonnes de la ligne affectée par la mise à jour prévue par l'événement.

```
CREATE OR REPLACE TRIGGER TrigSalaireDept
```

```
BEFORE INSERT ON employees
```

← Déclaration de l'événement déclencheur

```
FOR EACH ROW
```

```
DECLARE v_min_sal jobs.min_salary%TYPE;
```

```
v_max_sal jobs.max_salary%TYPE;
```

```
v_job_name jobs.job_title%TYPE;
```

```
BEGIN
```

← Corps du déclencheur

```
SELECT min_salary, max_salary, job_title INTO v_min_sal,  
v_max_sal, v_job_name FROM jobs WHERE job_id = :NEW.job_id;
```

```
IF :NEW.salary BETWEEN v_min_sal AND v_max_sal THEN NULL;
```

```
ELSE RAISE_APPLICATION_ERROR(-20200, 'Salaire ' || :NEW.salary  
|| ' hors grille pour ' || v_job_name || ' [' || v_min_sal ||  
'-' || v_max_sal || ']');
```

← Vérification du salaire

```
END IF;
```

```
EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-  
20201, 'Job inconnu : ' || :NEW.job_id);
```

```
WHEN OTHERS THEN RAISE;
```

```
END;
```

← Retour de l'erreur courante

Tests du déclencheur

Événement déclencheur	Sortie Oracle
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (300, 'Ali', 'Bensalem', 'ABENSALEM', SYSDATE, 'IT_PROG', 6000, 60);	1 ligne créée. SQL> SELECT employee_id, first_name, salary, job_id FROM employees WHERE employee_id = 300; EMPLOYEE_ID FIRST_NAME SALARY JOB_ID ----- 300 Ali 6000 IT_PROG
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (301, 'Sara', 'Idrissi', 'SIDRISSI', SYSDATE, 'IT_PROG', 500, 60);	ERREUR à la ligne 1 : ORA-20200: Salaire 500 hors grille pour Programmer [4000-10000] ORA-06512: à "HR.TRIGSALAIREDEPT", ligne 14 ORA-04088: erreur lors d'exécution du déclencheur 'HR.TRIGSALAIREDEPT'
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (302, 'Omar', 'Fassi', 'OFASSI', SYSDATE, 'XX_FAKE', 5000, 60);	ERREUR à la ligne 1 : ORA-20201: Job inconnu : XX_FAKE ORA-06512: à "HR.TRIGSALAIREDEPT", ligne 20 ORA-04088: erreur lors d'exécution du déclencheur 'HR.TRIGSALAIREDEPT'
☐ Insertion acceptée ☐ Insertion rejetée par le trigger	

La directive « : NEW »

- Le trigger **BEFORE INSERT** permet de vérifier, avant l'ajout d'un employé, que son salaire appartient bien à la fourchette salariale définie pour son poste.
- On utilise un déclencheur **FOR EACH ROW** car on désire qu'il s'exécute autant de fois qu'il y a de lignes concernées par l'événement déclencheur.
- Chaque enregistrement qui tente d'être ajouté dans la table **employees** est désigné par **:NEW** au niveau du code du déclencheur.

La directive « :OLD »

- Chaque enregistrement qui tente d'être supprimé d'une table qui inclut un déclencheur de type **DELETE FOR EACH ROW**, est désigné par **:OLD** au niveau du code du déclencheur.
- Programmons le déclencheur **TrigDelEmployee** qui surveille les suppressions de la table **Employees** et décrémente de 1 la colonne **nb_employees** pour le département concerné par la suppression.
- L'événement déclencheur est ici **AFTER DELETE** car il faudra s'assurer que la suppression n'est pas entravée par d'éventuelles contraintes référentielles.
- Le trigger ne s'exécute qu'après la confirmation que le **DELETE** a bien réussi. Si le **DELETE** échoue, le trigger ne tourne jamais.

La directive « :OLD »

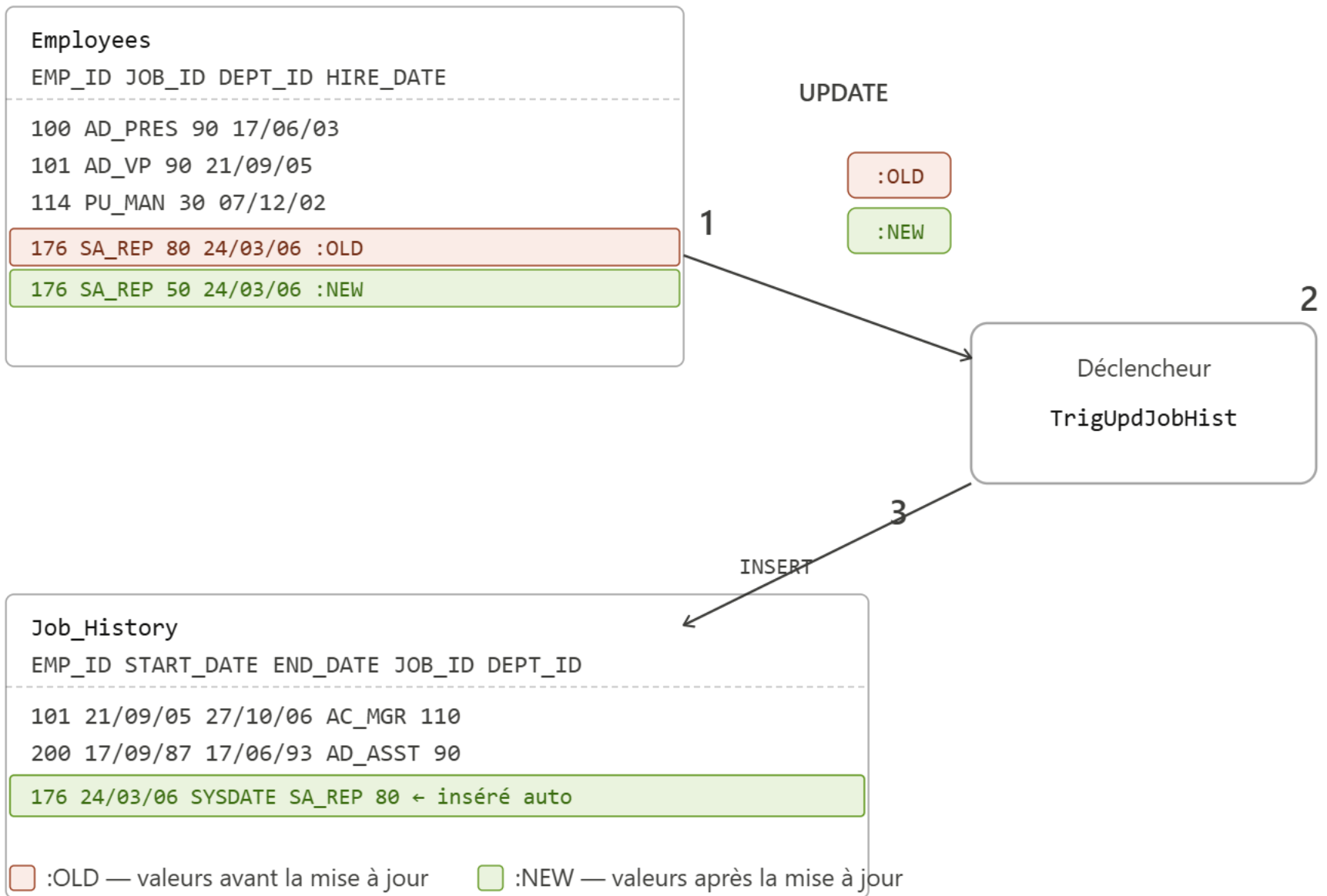
```
CREATE OR REPLACE TRIGGER TrigDelEmployee
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
UPDATE departments SET nb_employees=nb_employees-1
WHERE department_id =:OLD.department_id;
END;
/
```

Quand utiliser à la fois les directives :NEW et :OLD ?

- Seuls les déclencheurs de type **UPDATE FOR EACH ROW** permettent de manipuler à la fois les directives **:NEW** et **:OLD**.
- En effet, la mise à jour d'une ligne dans une table fait intervenir une nouvelle donnée qui en remplace une ancienne.
 - L'accès aux anciennes valeurs se fera par la notation pointée du pseudo-enregistrement :OLD
 - L'accès aux nouvelles valeurs se fera par :NEW

Les directives :NEW et :OLD

```
CREATE OR REPLACE TRIGGER TrigUpdJobHist
AFTER UPDATE OF department_id ON employees
FOR EACH ROW
BEGIN
    INSERT INTO job_history (employee_id, start_date,
end_date, job_id, department_id)
    VALUES (:OLD.employee_id,
            :OLD.hire_date,
            SYSDATE,
            :OLD.job_id,
            :OLD.department_id) ;
END ;
```

Les Déclencheurs de Lignes

Le principe

- ① UPDATE — On change le DEPARTMENT_ID de l'employé 176 (de 80 → 50). Le trigger voit :OLD (ancien département = 80) et :NEW (nouveau département = 50).
- ② Déclencheur *TrigUpdJobHist* — Il s'active automatiquement après chaque UPDATE sur EMPLOYEES.
- ③ INSERT automatique — Il archive l'ancienne affectation dans *JOB_HISTORY* en utilisant :OLD.DEPARTMENT_ID, :OLD.JOB_ID, et SYSDATE comme date de fin.

Gestion des déclencheurs

SQL	Commentaires
<code>ALTER TRIGGER <i>nomDéclencheur</i> COMPILE;</code>	Recompilation d'un déclencheur.
<code>ALTER TRIGGER <i>nomDéclencheur</i> DISABLE;</code>	Désactivation d'un déclencheur.
<code>ALTER TABLE <i>nomTable</i> DISABLE ALL TRIGGERS;</code>	Désactivation de tous les déclencheurs d'une table.
<code>ALTER TRIGGER <i>nomDéclencheur</i> ENABLE;</code>	Réactivation d'un déclencheur.
<code>ALTER TABLE <i>nomTable</i> ENABLE ALL TRIGGERS;</code>	Réactivation de tous les déclencheurs d'une table.
<code>DROP TRIGGER <i>nomDéclencheur</i>;</code>	Suppression d'un déclencheur.

Triggers

Exemple 1 :

- Écrire un trigger ***TrigProtectManager*** qui se déclenche **avant chaque suppression** d'un employé.
- Le trigger doit vérifier si l'employé est référencé comme manager_id dans la table DEPARTMENTS.
- Si c'est le cas, **bloquer la suppression** avec un message d'erreur explicite contenant l'identifiant de l'employé.

Triggers

```
CREATE OR REPLACE TRIGGER TrigProtectManager
BEFORE DELETE ON employees
FOR EACH ROW
DECLARE
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM departments
WHERE manager_id = :OLD.employee_id;
    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20400, 'Impossible : employé
        '|| :OLD.employee_id ||' est le manager d'un département);
    END IF;
END;
/
```

Triggers

Exemple 2 :

- Écrire un trigger **TrigDecrEmp** qui se déclenche **après chaque suppression** d'un employé.
- Ce trigger doit **décrémenter de 1** la colonne **nb_employees** du département concerné dans la table DEPARTMENTS.
- Il doit également **archiver** les informations de l'employé supprimé dans la table **employees_archive** avec la date du jour.

Triggers

```
CREATE OR REPLACE TRIGGER TrigDecrEmp
AFTER DELETE ON employees
FOR EACH ROW BEGIN
UPDATE departments SET nb_employees = nb_employees-1
WHERE department_id = :OLD.department_id;
INSERT INTO employees_archive (employee_id,
first_name, last_name, job_id, salary, department_id,
date_suppression)
VALUES (:OLD.employee_id, :OLD.first_name, :OLD.last_n
ame, :OLD.job_id, :OLD.salary, :OLD.department_id,
SYSDATE) ;
END ;
/
```

Triggers

Exemple 3 :

Lorsqu'une augmentation du prix unitaire (PU) d'un produit est tentée, il faut limiter l'augmentation à **10%** du prix en cours.

Triggers

```
UPDATE PRODUIT
SET Prix = 15.99
WHERE Codprod = 10 ;
```

```
CREATE OR REPLACE TRIGGER BORNER_AUGMENTPU
BEFORE UPDATE OF Prix ON PRODUIT
FOR EACH ROW
When (New.Prix > Old.Prix * 1.1)
BEGIN
    : New.Prix := :Old.Prix * 1.1 ;
END ;
/
```

	Codprod	Prix	
Ligne avant (OLD)	10	11	

	Codprod	Prix	
Ligne après (NEW)	10	15.99	

	Codprod	Prix	
Ligne après (NEW)	10	12.1	